2021 IEEE 10th Data Driven Control and Learning Systems Conference
May 14-16, 2021, Suzhou, China

Model-based Ensemble Reinforcement Learning with Soft Proximal Policy Optimization

Dazi Li*, Fuqiang Zhu

College of Information Science and Technology, Beijing University of Chemical Technology, Beijing 100029, PR China
E-mail: lidz@mail.buct.edu.cn

Abstract: At present, model-free reinforcement learning has been widely used in games, robot control and other fields, and has achieved good results. However, these algorithms require a large number of samples to achieve good performance, which limits the application of model-free reinforcement learning methods in real-world domains. Model-based reinforcement learning can use fewer samples, but requires careful tuning. In this article, we use a neural network ensemble model to learn the dynamics, and incorporate model predictive control as the basic control framework. The dynamic model is also used to train an initial model-free neural network to achieve a combination of sampling efficiency and performance. We evaluated our method on the MuJoCo experimental platform. The results show that, compared with other model-free and model-based methods, our approach achieves better task performance while having excellent sample efficiency.

Key Words: Reinforcement Learning, Model Predictive Control, Policy Gradient

2021 IEEE 10th Data Driven Control and Learning Systems Conference (DDCLS) | 978-1-6654-2423-3/21/$31.00 ©2021 IEEE | DOI: 10.1109/DDCLS52934.2021.9455658

1 Introduction

Reinforcement learning [1] algorithms have achieved success in many fields, including learning to play Atari games by inputting pictures [2,3], and robot control [4]. Most of these applications use model-free reinforcement learning algorithms and do not need to model the environment. As a branch of machine learning, deep reinforcement learning learns the optimal policy through trial and error to solve decision-making problems. There are three basic methods of reinforcement learning: the method based on Policy Evaluation [2,3], the method based on Policy Search [5] and the Actor-Critic method [6]. In these algorithms, an agent constantly interacts with the environment through trial and error, thereby learning a policy to maximize rewards from the environment. However, in the process of policy learning, these algorithms will suffer from high sample complexity and usually require millions of samples to obtain good performance. Their high sample complexity often limits their application to simulated domains. In real-world domains, sample collection is limited. Therefore, model-based reinforcement learning algorithms have more advantages than model-free reinforcement learning algorithms in terms of sample efficiency. Model-based reinforcement learning can first obtain a prediction model of the world through iteration, and then use the model to make decisions, which is generally regarded as more sampling efficiency.

However, learning an accurate environmental model is a challenging problem. Modeling errors weaken the effectiveness of these algorithms, leading to a policy that use the defects of the model, which is called model bias [7]. That is to say, these methods require a very strict form of the learning models, and the models need to be carefully tuned to make them applicable. Therefore, although extending a model-based algorithm to a deep neural network model sounds simple, the accuracy of the established model is an important factor affecting the efficiency of the algorithm. In low-dimensional problems where data efficiency is critical, Gaussian processes (GPs) are usually the model of choice in MBRL, and function approximators have also been widely used in MBRL [8,9]. Thanks to the development of deep learning, the use of neural networks to train environmental dynamic models has gradually become an important research direction. Unlike Gaussian processes, neural networks have the characteristics of adaptive learning and easy training in the state of big data, and may represent more complex functions. In the work of Nagabandi et al. [10], it was proved that the medium-sized neural network model can produce a stable and reasonable gait in the model-based reinforcement learning algorithm, thereby completing various complex motion tasks.

At the same time, like other machine learning methods such as deep learning, model-based deep reinforcement learning also needs to face such a problem: it not only needs to perform well in the existing training samples, but also needs to perform well for new samples that do not appear in the training samples, which is the so-called generalization problem. The challenge in reinforcement learning that is different from other learning problems is that there is always a contradiction between exploration and exploitation in reinforcement learning. To obtain more rewards, the reinforcement learning agent must choose those actions that can truly obtain greater reward value, but in order to discover these actions, the reinforcement learning agent must try some unknown actions. Reinforcement learning agents need to use actions that can get higher rewards. At the same time, they also need to explore unused actions in order to make better action choices in the future.

In order to solve these problems, we use an ensemble of deep neural networks to maintain the uncertainty and generalization of the model. During policy learning, each imagined step comes from ensemble predictions. We add random noise to increase the exploration of the policy. Using this technique, the policy can learn to become more robust to deal with various possible situations that may be encountered in the real environment.

In this paper, a new fusion model-based ensemble algorithm and improved model-free algorithm architecture

*This work is supported by National Natural Science Foundation (NNSF) of China under Grant 61873022.

is proposed. The experimental results show that the method proposed in this paper can achieve better results in comparative experiments.

2 Model-based Reinforcement Learning

The research of reinforcement learning is based on the Markov decision process (MDP) [11]. The reinforcement learning problem is modeled as a Markov decision process, defined by the following five-tuple (S, A, P, R, γ). S represents the environmental information or the states observed by the agent, A represents the action performed by the agent, the state transition probability P reflects that when the agent is in the environment state S, after taking action A, the environment moves to the next state S', R is the reward function determined by the environment, and γ is the discount factor with a value range of [0, 1].

At each time *t,* the agent observes the current environment state s_t ,chooses the action a_t according to the policy π, gets a reward r_t. Then the environment transfers to the next state s_{t+1} The goal of the agent is to maximize the total reward for future discounts.

From $t=0$ to $t=T$, the trajectory generated by the interaction between the agent and the environment is denoted as $\tau=(s_0,a_0,\ldots,a_{T-1},s_T)$, where T is the time horizon.

In model-based reinforcement learning (MBRL), the dynamic model is used to predict the future state of the environment, which is used to assist agent select actions. Let $f_\theta(s_t,a_t)$ denote the learned dynamic function, the parameters are denoted by θ. Then, the action selection can be described as the following optimization problem:

$$a_t = \arg\max_{a_t} \sum_{t=0}^{T} \gamma^t r(s_t, a_t) \tag{1}$$

It is usually necessary to solve this optimization problem at each time step, perform only the first action, and then re-plan with the updated state information at the next time step. This kind of control scheme is usually called model predictive control (MPC). It is well known that this mothed can compensate the errors in the model in time.

3 Proposed Algorithm Framework

3.1 Model Learning

First, we use a set of deep neural networks to build a dynamic model ensemble. When state s_{t+1} is too similar to state s_t and the effect of the action on the output seems to be small, it is difficult to learn the dynamic function. As the time between states becomes shorter, and the state difference cannot indicate the basic dynamics well. Like the methods in other works, our methods also use a given state and action as input to predict the change of the state, instead of the next state, and normalize the input and output of the neural network.

The objective of dynamics model learning is minimizing the loss:

$$loss_{dynamics} = \sum_{(s_t,a_t,s_{t+1})\in D} \frac{1}{2}\left\|(s_{t+1}-s_t)-f_\theta(s_t,a_t)\right\|^2 \tag{2}$$

where D is the training dataset of the transition experienced by the agent. We use the Adam optimizer to train this dynamics function.

Let B to denote the number of ensemble models, and let function f_{θ_b} to denote as the b^{th} network with the parameter θ_b. Give state s_t ,action a_t and the next state s_{t+1} tuple from the dataset D, where t represents the time, and train each network by minimizing the Mean Square Error (MSE) loss. The same dataset D is used to train all models, thereby reducing model errors caused by accidents that may exist when only one model is used, and improving the accuracy of the ensemble models. After training each model, we use the Gaussian distribution as the prediction distribution.

$$\mu = \frac{1}{B}\sum_{b=1}^{B} f_{\theta b} \tag{3}$$

$$\Sigma = \frac{1}{B}\sum_{b=1}^{B}\left(f_{\theta b}-\mu\right)^2 \tag{4}$$

Formula (3) and formula (4) are the mean and variance formulas of the final dynamic model, respectively. In this paper, B is set to 5.

3.2 Model Predictive Control

Since the model ensemble f_θ has been learned, it can be uses for control by predicting the future outcome of the policy π or action a, and then select the specific action that predicts the highest return.

Given the state s_t of the system at time t, the prediction horizon T of the MPC controller, and an action sequence $A_t^T = \{a_t, \ldots, a_{t+T}\}$, the dynamics model ensemble f_θ predicts the possible trajectory $S_t^T=\{s_t, \ldots, s_{t+T}\}$. At each time step t, the MPC controller applies the first action in the best action sequence. Random Shooting [12] sampling method is use to generate action sequences. It is widely used because of its parallelism and ease of implementation.

We collect random trajectories into the dataset D, and then trains a neural network policy $\pi(s)$ through supervised learning to match these trajectories. We parameterize policy π as a Gaussian policy $\pi(s) \sim N\left(\mu_\varphi, \Sigma_{\pi_\varphi}\right)$, where the mean is parameterized by a neural network μ_φ , and the covariance is Σ_{π_φ} . Use formula (5) to train the parameters of this policy:

$$\min \frac{1}{2}\sum_{(s_t,a_t)\in D}\left\|a_t-\mu_\varphi(s_t)\right\|^2 \tag{5}$$

3.3 DAgger Optimization

In order to achieve better performance, DAgger [13] method is applied in this work: in the set number of iterations, the policy obtained from the previous training interacts with the environment to continuously collect new training data, add these data to the dataset and optimize training in the entire dataset. The obtained policy network

model can have better generalization performance and fitting ability. After DAgger iterations, the policy $\pi(s)$ is used as the initial policy π_φ of the model-free reinforcement learning algorithm. In other words, the policy π_φ of the model-free algorithm is transferred from the model-based algorithm in which the policy $\pi(s)$ is trained in the form of supervised learning. Algorithm 1 and Fig. 1 show the process of our model-based training phase.

Algorithm 1 Model-based Training

```
initialize an empty dataset D
initialize a dynamic model ensemble fθ and a policy πφ
collect samples randomly from the environment and add them to D
train the model ensemble fθ and the policy πφ using D
for DAgger_iter = 1 to max_iter do
    get agent's state st
    use fθ to estimate optimal action sequence
    execute the first action in the selected action sequence and add them to D
    update the policy πφ
end for
```

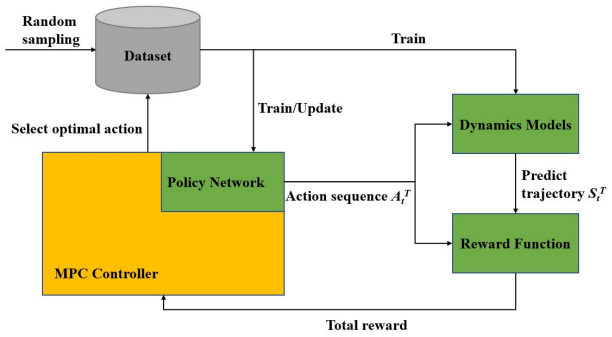

Fig. 1: Illustration of Algorithm 1.

3.4 Soft Proximal Policy Optimization

In the model-free training phase, we use Proximal Policy Optimization (PPO) [14] to continue to optimize the policy network. PPO algorithm is simple to implement and has excellent performance which is a common choice for benchmarking tasks. There are two primary variants of PPO: PPO-Penalty and PPO-Clip. PPO-Clip uses the following objective function:

$$L^{CLIP}(\varphi) = E[min(r_t(\varphi)A_t, clip(r_t(\varphi), 1-\ , 1+\ )A_t)] \quad (6)$$

where A_t is an advantage estimation. PPO is motivated by the same problems as Trust-Region Policy Optimization TRPO [15]: using the data currently possessed to take the biggest possible improvement steps in the policy without going too far, so as not to cause performance degradation. TRPO tries to use a complex second-order method to solve this problem, while PPO uses a first-order method to make the new policy closer to the old one. PPO uses pruning technology to approximate TRPO objective function, making the optimization problem in PPO not only easier to solve, but also similar to TRPO performance.

Our work is inspired by the idea of maximum entropy in the Soft Actor-Critic (SAC) [16] algorithm. The goal is to make the agent maximize the expected return while maximizing entropy. In other words, to perform tasks as randomly as possible when completing tasks, this method introduces a soft policy iterative algorithm, which alternates policy evaluation and policy improvement within the framework of maximum entropy. We call it Soft Proximal Policy Optimization (SPPO). In the policy evaluation step of the soft policy iteration, for the policy network, the soft state value function is introduced:

$$V(s_t) = E_{at\sim\pi}\left[Q(s_t, a_t) - \alpha log\ \pi(a_t \mid s_t)\right] \quad (7)$$

Then, we take the opposite of it as the cost loss, as shown in the formula (8):

$$L^{cost} = E_{at\sim\pi}\left[\alpha log\ \pi(a_t \mid s_t) - Q(s_t, a_t)\right] \quad (8)$$

So that the loss function is modified to formula (9):

$$L(\varphi) = L^{CLIP} + L^{cost} \quad (9)$$

In the policy improvement step, the policy is updated to the direction of the new loss function to ensure that this particular update option can improve the policy based on its soft value.

4 Experiment

4.1 Experimental Details

We validate our method on OpenAI MuJoCo [17] physical environment. The agent we use is Half Cheetah, as Fig. 2 shows.

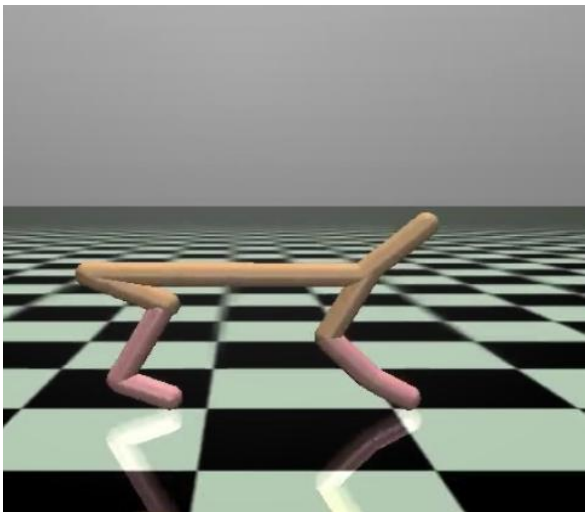

Fig. 2: Half Cheetah Environments

Half Cheetah is a 2D robot composed of 7 rigid links with 2 legs and 1 torso. Each of the 6 joints has an actuator. The observation includes the positions and velocities of all joints, except for the x position of the root joint. The specific settings of the environment we use is from Wang et al. [18]. The goal is to run as fast as possible while keeping the control input small. After each action is executed, the

environment will give the agent a reward of -0.1 times, which is intended to remind the agent to reduce the number of action execution as much as possible, so as to achieve the training goal of the agent. In short, the reward is the x direction speed plus the loss of control input.

First, interact with the environment through random actions to generate the state transition trajectories (s, a, s', r) required for pre-training, store them in the dataset D, and train the dynamic network. The learning rate of dynamics model training is 0.0003. Then use the DAgger method to use the dynamic model obtained and train the policy with 0.0001 learning rate. The policy model is iteratively trained in 7000 timesteps and an optimized policy network is obtained. After that, in the model-free phase, the SPPO algorithm is used to train the policy network for the second time. Our dynamics network has two hidden layers, each of dimension 512, with ReLU activations. And policy network use tanh activations and two hidden layers, each of dimension 64. In order to improve the robustness, we added noise to the action during the training session. we use Adam optimization algorithm for training, and the max timesteps are 200000.

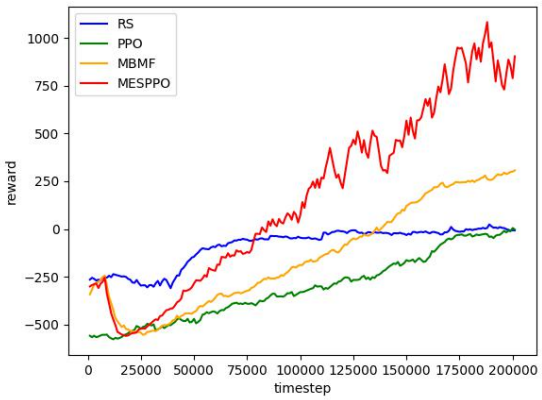

Fig. 3: Reward learning curves

4.2 Experimental Result Analysis

Our algorithm (MESPPO) is compared with Random Shooting (RS), PPO, and MBMF [10]. In all comparative experiments, these algorithms use the same hyperparameters. Fig. 3 shows reward learning curves of the four methods in the Half Cheetah task.

It can be seen from the Fig. 3 that the method we proposed can achieve the best results under the same experimental parameters. The randomness of the RS algorithm is too high to realize the optimization of the policy. The rewards of other algorithms generally increase as the training time increases. Under the same number of time steps, the reward obtained by the PPO algorithm is less than those of the model-based algorithm. In other words, PPO algorithm requires more training time to obtain higher rewards, which once again proves that the model-based algorithm is more efficient than the model-free algorithm. It also reflects the role of the environment model in providing guidance and assistance to the learning and optimization of reinforcement learning algorithms. It is true that in the transition phase between model-based training and model-free training, the rewards obtained by the agent have significantly decreased. Compared to general model-free algorithms, model-based algorithms can provide better initialization effects and reduce the time consumption of tuning caused by completely random initialization. Our method also surpasses the MBMF algorithm, which is also a model-based algorithm. Due to the introduction of entropy regularization terms, our proposed algorithm is more exploratory than the MBMF algorithm in the model-free tuning stage, and the choice of actions is more random, so as to explore more unknown and meaningful actions. Therefore, the agent obtains a higher reward return.

In order to reduce the experimental error caused by chance, each of the methods are run three times under the same experimental conditions, and the average rewards of the three times are taken. The experimental results are shown in Table 1.

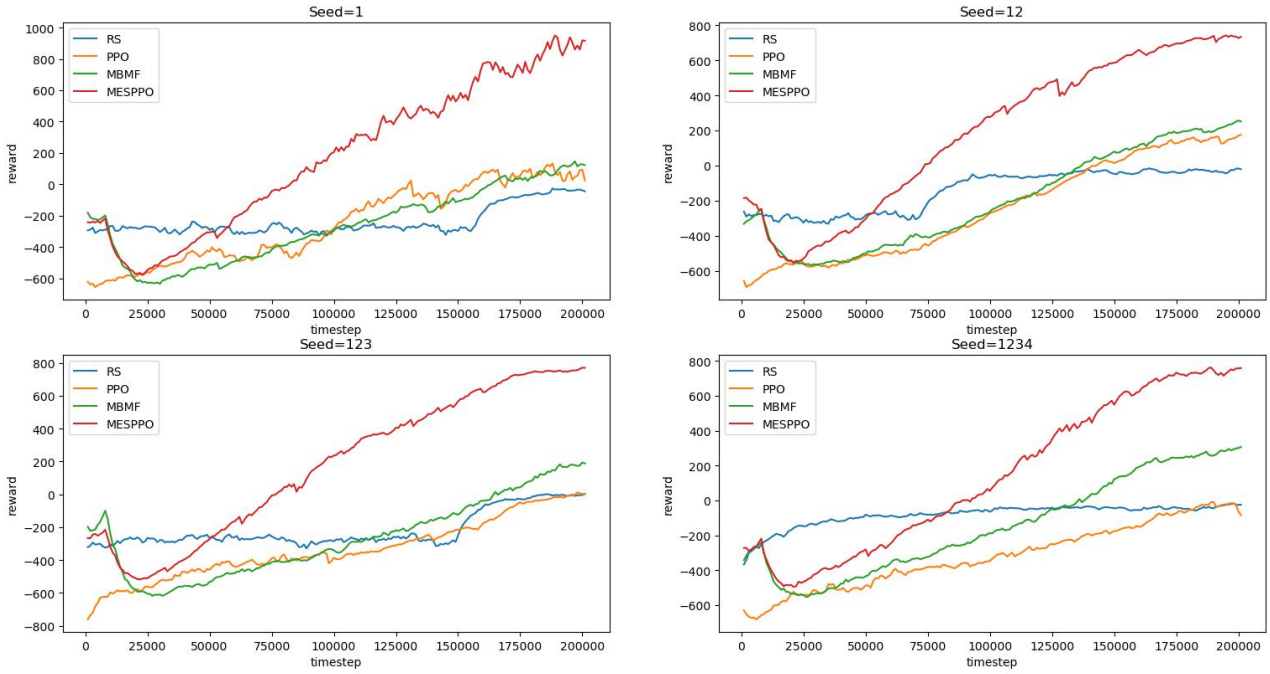

Fig. 4: Reward learning curves of four methods under different seeds

Table 1: Comparation of average reward

Methods	Reward1	Reward2	Reward3	Average reward
RS	-9.2	-74.4	-22.1	-35.2
PPO	-281.6	83.2	-46.6	-81.7
MBMF	354.2	159.3	79.4	197.6
MESPPO	789.5	897.9	653.8	780.4

Table 1 shows that our algorithm has achieved significantly higher scores compared with other methods in the Half Cheetah experimental environment. The proposed algorithm is compared with other algorithms under different random seed numbers, as shown in Fig. 4. Experimental results show that our algorithm has good generalization performance. Under different random seeds, it can obtain higher rewards than other algorithms.

In addition, an experiment is conducted to compare the experimental results of one network and five ensemble networks, in order to verify the role of ensemble learning. Except for the number of networks, the other hyperparameters are exactly the same. The results of comparison are shown in Table 2.

Table 2: Comparation of reward and running time

Number of networks	Average reward	Running time
1	180.6	30min
5-ensemble	254.3	31min

The experiment compares two aspects of running time and average reward. It can be seen from Table 2 that due to the parallelization setting, although the network is increased from 1 to 5, the running time of the entire experiment does not increase too much. And because the multi-network ensemble learning reduces the problem of overfitting and model errors, and improves the robustness and accuracy of the dynamic model, the environment model is better, and the experimental results is also significantly improved.

5 Conclusions

In this work, a simple and powerful model-based reinforcement learning algorithm is proposed, which can use a small number of samples to learn the neural network dynamics function of complex simulated motion tasks. In our proposed method, we use ensemble learning methods to avoid the problem of model overfitting. While improving the accuracy of the model, it does not consume too much time cost. At the same time, the soft policy iterative algorithm is combined with Proximal Policy Optimization method, and achieved a good level of performance.

References

[1] R. Sutton, and A. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA: MIT Press, 2018.

[2] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, Playing Atari with Deep Reinforcement Learning, *Computer Science*, 2013.

[3] V. Mnih, K. Kavukcuoglu, D. Silver, A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. Fidjeland, G. Ostro-vski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, Human-level control through deep reinforcement learning, *Nature*, 518(7540):529–533, 2015.

[4] T. Lillicrap, J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, Continuous control with deep reinforcement learning, *CoRR*, abs/1509.02971, 2015.

[5] M. Deisenroth, G. Neumann, and J. Peters, *A Survey on Policy Search for Robotics*, now, 2013.

[6] V. Mnih, A. Badia, M. Mirza, A. Graves, T. Harley, T. Lillicrap and K. Kavukcuoglu, Asynchronous Methods for Deep Reinforcement Learning, in *Proceedings of the 33nd International Conference on Machine Learning*, 2016: 1928–1937.

[7] M. Deisenroth, and C. Rasmussen, PILCO: A Model-Based and Data-Efficient Approach to Policy Search, in *Proceedings of the 28th International Conference on machine learning*, 2011: 465–472.

[8] T. Kurutach, I. Clavera, Y. Duan, A. Tamar, and P. Abbeel, Model-Ensemble Trust-Region Policy Optimization, *arXiv preprint arXiv:1802.10592*, 2018.

[9] G. Williams, N. Wagener, B. Goldfain, P. Drews, J. Rehg, B. Boots, and E. Theodorou, Information theoretic MPC for model-based reinforcement learning. *IEEE International Conference on Robotics and Automation*, 2017: 1714–1721.

[10] A. Nagabandi, G. Kahn, R. Fearing, and S. Levine, Neural Network Dynamics for Model-Based Deep Reinforcement Learning with Model-Free Fine-Tuning, in *Proceedings of IEEE International Conference on Robotics and Automation,* 2018: 7579–7586.

[11] C. Innes, and A. Lascarides, Learning Factored Markov Decision Processes with Unawareness, *arXiv preprint arXiv: 1902.10619*, 2019.

[12] A. Rao, A survey of numerical methods for optimal control, *Advances in Astronautical Sciences*, 135(1):497–528, 2009.

[13] S. Ross, G. Gordon, and J. Bagnell, A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning, in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, 2011.

[14] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, Proximal Policy Optimization Algorithms, *arXiv preprint arXiv:1707.06347*, 2017.

[15] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, Trust Region Policy Optimization, in *Proceedings of the 32nd International Conference on Machine Learning*, 2015: 1889–1897.

[16] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor, *arXiv preprint arXiv:1801.01290*, 2018.

[17] E. Todorov, T. Erez, and Y. Tassa, MuJoCo: A physics engine for model-based control, *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012: 5026–5033.

[18] T. Wang, X. Bao, I. Clavera, J. Hoang, Y. Wen, E. Langlois, S. Zhang, G. Zhang, P. Abbeel, and J. Ba, Benchmarking Model-Based Reinforcement Learning, *arXiv preprint arXiv:1907.02057*, 2019.